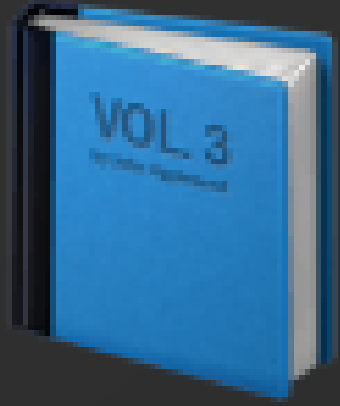




Lesson: Functions in Python

Introduction

As your programs grow, you will often find yourself writing the same code multiple times.
For example:

- Printing a greeting message
- Calculating a value
- Processing user input

Instead of repeating code, Python allows you to organize your program into **functions**.
A **function** is a reusable block of code that performs a specific task.

👉 Functions help you:

- Avoid repeating code
- Make programs easier to read
- Break problems into smaller pieces

In this lesson, you will learn how to create and use functions in Python.

Why Functions Are Important

Functions are one of the most important tools in programming.

Without Functions (Repetition)



```
print>Hello, Eric)
print>Welcome to the course)
print>Hello, Sarah)
print>Welcome to the course)
```

This works, but it repeats the same logic.

With Functions (Reusable Code)



```
def greet(name):  
    print(Hello,, name)  
    print>Welcome to the course)  
greet(Eric)  
greet(Sarah)
```

👉 Now the logic is written once and reused.

Benefits of Functions

- **Reusability** — write once, use many times
- **Readability** — code is easier to understand
- **Maintainability** — easier to fix or update
- **Organization** — programs are structured into logical pieces



Defining a Function

In Python, you define a function using the `def` keyword.



Example 1 — Simple Function

```
def say_hello():  
    print(Hello, World!)
```

Calling the Function

```
say_hello()
```

Output

```
Hello, World!
```

Explanation

- `def` starts the function definition
- `say_hello` is the function name
- `()` means the function takes no input
- The indented code runs when the function is called



Functions with Parameters

Functions become more powerful when they accept input.



Example 2 — Function with a Parameter

```
def greet(name):  
    print(Hello,, name)
```

Calling the Function

```
greet(Eric)  
greet(Alice)
```

Output

```
Hello, Eric  
Hello, Alice
```

Explanation

- `name` is a **parameter**
- It allows the function to work with different values
- Each time you call the function, you can pass a new value



Functions That Return Values

Functions can also send data back using the `return` statement.



Example 3 — Returning a Value

```
def add_numbers(a, b):  
    return a + b
```

Using the Function

```
result = add_numbers(3, 5)  
print(result)
```

Output

8

Explanation

- `return` sends a value back to the caller
- The result can be stored in a variable
- This allows functions to be used in calculations



Functions vs Print

It's important to understand the difference:

```
def add_numbers(a, b):  
    print(a + b)
```

VS



```
def add_numbers(a, b):  
    return a + b
```

👉 The first version:

- Displays the result
- Cannot be reused easily in other calculations

👉 The second version:

- Returns the result
- Can be reused in other parts of the program



Example 4 — Combining Input and Functions

```
def greet_user():  
    name = input(Enter your name: )  
    print>Hello,, name)  
greet_user()
```

What This Does

- Prompts the user for input
- Uses a function to organize the logic
- Keeps the main program clean



Example 5 — Function with Multiple Parameters

```
def introduce(name, goal):  
    print>Hello,, name)  
    print>Your goal is to learn, goal)  
introduce(Eric, Python)  
introduce(Sarah, Cloud Computing)
```

Output



```
Hello, Eric  
Your goal is to learn Python  
Hello, Sarah  
Your goal is to learn Cloud Computing
```

Example 6 — Real-World Style Function

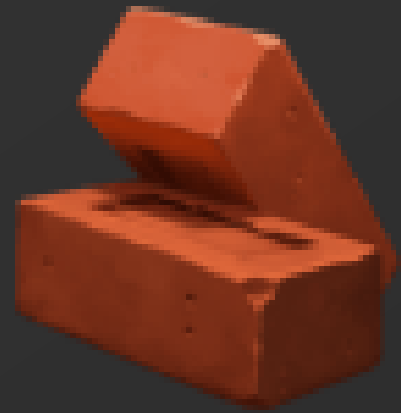
```
def calculate_total(price, tax_rate):  
    total = price + (price * tax_rate)  
    return total  
total_price = calculate_total(100, 0.07)  
print('Total price:', total_price)
```

Output

`Total price: 107.0`

What This Demonstrates

- Performing a calculation inside a function
- Returning a result
- Using that result in the main program



Building Programs with Functions

As programs grow, functions help structure your code.



Example — Organized Program



```
def get_name():  
    return input(Enter your name: )  
def get_goal():  
    return input(What do you want to learn? )  
def display_message(name, goal):  
    print>Hello,, name)  
    print(It's great that you want to learn, goal)  
name = get_name()  
goal = get_goal()  
display_message(name, goal)
```

Why This Is Better

- Each function has a single purpose
- The program is easier to read
- Each part can be tested separately

Common Beginner Mistakes

- Forgetting to call the function
- Missing parentheses `()`
- Incorrect indentation
- Using `print` instead of `return` when a value is needed

Summary

In this lesson, you learned:

- What functions are and why they are important
- How to define a function using `def`
- How to pass data using parameters
- How to return values using `return`
- How functions improve code organization and readability

Final Thought

“

Functions are how real programs are built.

They allow you to:

- Break problems into smaller steps
- Reuse code
- Build larger, more complex systems

👉 Mastering functions is a major step toward becoming a professional developer.